LangChain — Core Concepts

Framework • Prompts • Chains • Tools • RAG Stack • Retrievers

---

Part 1 — LangChain Framework Overview

LangChain is a framework for building applications with LLMs. Its core idea is to provide a unified interface across different model providers (OpenAI, Anthropic, Hugging Face, etc.) and a set of composable building blocks — prompts, models, parsers, chains, tools, memory, and retrievers — that snap together to build complex LLM pipelines.

The main building blocks are Models, Prompts, and Output Parsers. These three together form the basic unit of any LangChain application.

1.1 Models

LangChain wraps different LLM providers behind a common interface so you can swap models without rewriting your pipeline.

Chat Models (most common)

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_google_genai import ChatGoogleGenerativeAI

# All use the same interface
llm = ChatOpenAI(model='gpt-4o', temperature=0.7)
llm = ChatAnthropic(model='claude-3-5-sonnet-20241022')
llm = ChatGoogleGenerativeAI(model='gemini-1.5-pro')

# Invoke the model
response = llm.invoke('What is machine learning?')
print(response.content)
```

Chat models take messages and return a message. The unified .invoke() interface works the same regardless of provider — swap one line to switch models.

LLM Models (legacy — text in, text out)

```
from langchain_openai import OpenAI
llm = OpenAI(model='gpt-3.5-turbo-instruct')
response = llm.invoke('Explain neural networks in one line')
print(response)   # returns plain string, not message object
```

Embeddings Models

```
from langchain_openai import OpenAIEmbeddings
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vector = embeddings.embed_query('What is RAG?')
print(len(vector))   # 1536 dimensions
```

1.2 Prompts

LangChain separates prompt construction from model invocation. Instead of building strings manually, you define prompt templates that can be reused, versioned, and tested.

Message Types

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

messages = [
    SystemMessage(content='You are a helpful data science tutor.'),
    HumanMessage(content='Explain overfitting in simple terms.'),
    AIMessage(content='Overfitting is when...'),   # for few-shot
    HumanMessage(content='Give me an example.'),
]
```

```
response = llm.invoke(messages)
```

1.3 Output Parsers

By default, LLMs return raw text. Output parsers transform that text into structured Python objects — strings, lists, JSON, Pydantic models, etc.

```
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.output_parsers import CommaSeparatedListOutputParser

# StrOutputParser — extract just the string from AIMessage
parser = StrOutputParser()
chain = llm | parser
result = chain.invoke('List 3 ML algorithms')
print(type(result))   # <class 'str'>

# CommaSeparatedListOutputParser
parser = CommaSeparatedListOutputParser()
chain = prompt | llm | parser
result = chain.invoke({'topic': 'ML'})
print(result)   # ['Linear Regression', 'Decision Tree', 'Random Forest']
```

Pydantic Output Parser — structured JSON

```
from pydantic import BaseModel, Field
from langchain_core.output_parsers import PydanticOutputParser

class MovieReview(BaseModel):
    title: str = Field(description='movie title')
    rating: float = Field(description='rating from 1 to 10')
    summary: str = Field(description='one-line summary')

parser = PydanticOutputParser(pydantic_object=MovieReview)

prompt = PromptTemplate(
    template='Review this movie: {movie}\n{format_instructions}',
    input_variables=['movie'],
    partial_variables={'format_instructions': parser.get_format_instructions()}
)
chain = prompt | llm | parser
result = chain.invoke({'movie': 'Inception'})
print(result.title, result.rating)   # typed Python object
```

Part 2 — Prompt Templates

Prompt templates are reusable prompt structures with placeholders for dynamic values. Instead of writing string format code everywhere, you define the template once and fill it in at runtime.

2.1 Static Text Prompt

The simplest case — just a fixed string passed to the model.

```
from langchain_core.messages import HumanMessage

# Static — same every time
response = llm.invoke('What is the difference between AI and ML?')
```

2.2 Dynamic Prompts with f-strings (not recommended)

You can build prompts with plain Python f-strings, but this has no validation, no reusability, and is hard to test.

```
topic = 'overfitting'
level = 'beginner'
prompt = f'Explain {topic} to a {level}. Keep it under 100 words.'
response = llm.invoke(prompt)
```

This works but doesn't integrate with LangChain's chain system and can't be stored, versioned, or shared easily.

2.3 PromptTemplate Class

```
from langchain_core.prompts import PromptTemplate

# Define template with {placeholders}
template = PromptTemplate(
    template='Explain {topic} to a {level} in under 100 words.',
    input_variables=['topic', 'level']
)

# Fill in values
filled = template.invoke({'topic': 'neural networks', 'level': 'beginner'})
print(filled)   # StringPromptValue object

# Use in a chain
chain = template | llm | StrOutputParser()
result = chain.invoke({'topic': 'overfitting', 'level': 'expert'})
```

2.4 ChatPromptTemplate (for chat models)

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a {role}. Answer concisely.'),
    ('human',  'Explain {topic} in simple terms.'),
])

chain = prompt | llm | StrOutputParser()
result = chain.invoke({'role': 'data scientist', 'topic': 'gradient descent'})
```

2.5 Few-shot Prompt Template

```
from langchain_core.prompts import FewShotPromptTemplate

examples = [
    {'input': 'happy', 'output': 'sad'},
    {'input': 'tall',  'output': 'short'},
]

example_prompt = PromptTemplate(
    input_variables=['input', 'output'],
    template='Word: {input}\nAntonym: {output}'
)

few_shot = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    suffix='Word: {input}\nAntonym:',
    input_variables=['input']
)
chain = few_shot | llm | StrOutputParser()
print(chain.invoke({'input': 'fast'}))   # slow
```

2.6 Partial Variables

You can pre-fill some variables in a template and leave others for later. Useful for system-level context that's always the same.

```
prompt = PromptTemplate(
    template='Today is {date}. Answer this: {question}',
    input_variables=['date', 'question']
)

# Partially fill date
from datetime import datetime
partial_prompt = prompt.partial(date=datetime.now().strftime('%Y-%m-%d'))
```

```
# Only need question now
chain = partial_prompt | llm | StrOutputParser()
chain.invoke({'question': 'What day is it?'})
```

Part 3 — Chains

Chains connect multiple components into a pipeline. Modern LangChain uses LCEL (LangChain Expression Language) — the pipe operator | — to compose chains. Think of it like Unix pipes: data flows left to right, each component transforms it.

3.1 Simple Chain (LCEL)

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template('Summarize: {text}')

# Chain: prompt → model → parser
chain = prompt | llm | StrOutputParser()

result = chain.invoke({'text': 'LangChain is a framework for LLMs...'})
```

The | operator connects components. Each component's output becomes the next component's input. This is the core of LCEL.

3.2 Sequential Chain

Run multiple chains in sequence — output of chain 1 becomes input of chain 2.

```
# Chain 1: translate to French
translate_prompt = ChatPromptTemplate.from_template(
    'Translate this to French: {text}'
)
translate_chain = translate_prompt | llm | StrOutputParser()

# Chain 2: summarize the translation
summarize_prompt = ChatPromptTemplate.from_template(
    'Summarize this French text in one sentence: {french_text}'
)
summarize_chain = summarize_prompt | llm | StrOutputParser()

# Connect them: pass french_text explicitly
from langchain_core.runnables import RunnablePassthrough

full_chain = (
    {'french_text': translate_chain}
    | summarize_chain
)
result = full_chain.invoke({'text': 'Machine learning is a subset of AI...'})
```

3.3 Parallel Chains (RunnableParallel)

Run multiple chains at the same time on the same input and collect all results. Useful for generating multiple outputs simultaneously.

```
from langchain_core.runnables import RunnableParallel

pros_chain   = ChatPromptTemplate.from_template('List pros of {tech}')   | llm |
StrOutputParser()
cons_chain   = ChatPromptTemplate.from_template('List cons of {tech}')   | llm |
StrOutputParser()
uses_chain   = ChatPromptTemplate.from_template('List use cases of {tech}') | llm |
StrOutputParser()

parallel = RunnableParallel({
    'pros':  pros_chain,
```

```
    'cons':  cons_chain,
    'uses':  uses_chain,
})

result = parallel.invoke({'tech': 'LangChain'})
print(result['pros'])   # pros string
print(result['cons'])   # cons string
# All three chains ran concurrently
```

3.4 Conditional Routing Chain

Route the input to different chains based on a condition. Like an if/else in your pipeline.

```
from langchain_core.runnables import RunnableBranch, RunnableLambda

python_chain  = ChatPromptTemplate.from_template('Answer this Python question:
{question}') | llm | StrOutputParser()
sql_chain     = ChatPromptTemplate.from_template('Answer this SQL question: {question}')
| llm | StrOutputParser()
general_chain = ChatPromptTemplate.from_template('Answer this question: {question}')
| llm | StrOutputParser()

# Classifier — decides which chain to use
classify_prompt = ChatPromptTemplate.from_template(
    'Classify this question as python, sql, or general (one word only): {question}'
)
classifier = classify_prompt | llm | StrOutputParser()

branch = RunnableBranch(
    (lambda x: 'python' in x['topic'].lower(), python_chain),
    (lambda x: 'sql'    in x['topic'].lower(), sql_chain),
    general_chain   # default
)

full_chain = RunnablePassthrough.assign(topic=classifier) | branch
result = full_chain.invoke({'question': 'How do list comprehensions work?'})
```

3.5 Lambda Functions in Chains

RunnableLambda wraps any Python function into a chain step. Use it for data transformation between steps.

```
from langchain_core.runnables import RunnableLambda

# Simple transformation
upper = RunnableLambda(lambda x: x.upper())
chain = prompt | llm | StrOutputParser() | upper

# More complex — extract just first sentence
first_sentence = RunnableLambda(lambda text: text.split('.')[0] + '.')
chain = prompt | llm | StrOutputParser() | first_sentence

# Transform dict between chains
reshape = RunnableLambda(lambda x: {'summary': x, 'word_count': len(x.split())})
chain = prompt | llm | StrOutputParser() | reshape
```

Part 4 — Tools & LangSmith

4.1 Calling Tools Without LangChain

Before LangChain tools, you'd call an external function manually and manually feed the result back to the LLM. This requires you to parse the LLM's intent, call the function, and format the result — all by hand.

```
import json, requests

# Manual tool call (no LangChain)
response = openai_client.chat.completions.create(
```

```
    model='gpt-4o',
    messages=[{'role':'user','content':'What is the weather in London?'}],
    tools=[{'type':'function','function':{'name':'get_weather','parameters':{...}}}]
)

# Check if model wants to call a tool
if response.choices[0].finish_reason == 'tool_calls':
    tool_call = response.choices[0].message.tool_calls[0]
    args = json.loads(tool_call.function.arguments)
    result = get_weather(**args)   # manually call the function
    # manually add result back to messages and call again
```

This is verbose and error-prone. You have to handle the tool call loop manually.

4.2 Calling Tools With LangChain

LangChain wraps all of this — define a tool with @tool, bind it to the model, and LangChain handles the call-response loop automatically via an agent.

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent

# Define a tool with @tool decorator
@tool
def get_weather(city: str) -> str:
    '''Get current weather for a city.'''
    return f'The weather in {city} is 22°C and sunny.'

@tool
def calculate(expression: str) -> float:
    '''Evaluate a math expression.'''
    return eval(expression)

llm = ChatOpenAI(model='gpt-4o')
tools = [get_weather, calculate]

# Create agent — handles tool call loop automatically
agent = create_react_agent(llm, tools)

result = agent.invoke({
    'messages': [('user', 'What is the weather in Paris and what is 15 * 7?')]
})
print(result['messages'][-1].content)
```

The @tool decorator reads the function signature and docstring to generate the tool schema the LLM needs. LangChain handles the loop: model decides to call a tool → tool runs → result fed back → model continues.

4.3 LangSmith — Observability

LangSmith is LangChain's observability platform. Think of it like n8n for debugging — it gives you a visual trace of every step in your chain or agent: what prompt went in, what came out, how long each step took, and how many tokens were used. Essential for debugging complex chains.

Setup

```
# Set environment variables
import os
os.environ['LANGCHAIN_TRACING_V2'] = 'true'
os.environ['LANGCHAIN_API_KEY']    = 'your-langsmith-api-key'
os.environ['LANGCHAIN_PROJECT']    = 'my-project-name'

# That's it — no code changes needed
# All chain.invoke() calls are automatically traced
```

Once tracing is on, every invocation is logged to LangSmith. You can open the UI and see the full trace tree.

What LangSmith shows you

- Full input/output at every chain step — exactly what prompt was sent to the LLM

- Latency per step — identify which part of your chain is slow
- Token usage and cost per run
- Error traces with full context when something fails
- Side-by-side comparison of different prompt versions
- Datasets and automated evaluation — test your chain against a set of examples

Adding custom metadata

```
chain.invoke(
    {'question': 'What is RAG?'},
    config={'metadata': {'user_id': 'u123', 'session': 'abc'}}
)
```

Metadata shows up in LangSmith traces, making it easy to filter and group runs by user or session.

Part 5 — Document Loaders & Text Splitters

Before you can do RAG (retrieval-augmented generation), you need to load your documents and split them into chunks small enough to embed and retrieve. LangChain has loaders for almost every file format and multiple chunking strategies.

5.1 Document Loaders

Loaders read raw content from different sources and return a list of Document objects (content + metadata).

```
from langchain_community.document_loaders import (
    PyPDFLoader, TextLoader, CSVLoader,
    WebBaseLoader, UnstructuredWordDocumentLoader
)

# PDF
loader = PyPDFLoader('document.pdf')
docs = loader.load()          # list of Document objects, one per page

# Plain text
loader = TextLoader('notes.txt', encoding='utf-8')
docs = loader.load()

# Web page
loader = WebBaseLoader('https://docs.langchain.com')
docs = loader.load()

# CSV — each row becomes a document
loader = CSVLoader('data.csv')
docs = loader.load()

# Each Document has .page_content (string) and .metadata (dict)
print(docs[0].page_content[:200])
print(docs[0].metadata)    # {'source': 'document.pdf', 'page': 0}
```

5.2 Text Splitters — Chunking Strategies

You can't embed an entire document — you split it into chunks, embed each chunk, and store them. The chunking strategy heavily affects retrieval quality.

Character Text Splitter

Splits on a single character (default is newline). Simple but can cut mid-sentence.

```
from langchain_text_splitters import CharacterTextSplitter

splitter = CharacterTextSplitter(
    separator='\n',
    chunk_size=1000,      # max chars per chunk
    chunk_overlap=200     # overlap between chunks to preserve context
)
chunks = splitter.split_documents(docs)
```

Recursive Character Text Splitter (recommended)

Tries to split on a hierarchy of separators: paragraph → sentence → word → character. Always results in semantically cleaner chunks than simple character splitting. This is the default choice for most use cases.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=['\n\n', '\n', '. ', ' ', '']   # tries in order
)
chunks = splitter.split_documents(docs)
```

It first tries to split on double newline (paragraph). If any chunk is still too big, it tries single newline, then sentence, then word, then character. This produces the most natural splits.

Semantic Chunking

Instead of splitting by character count, splits by meaning — combines sentences until adding the next sentence would make the chunk semantically too different. More intelligent but slower and requires embeddings at chunking time.

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

splitter = SemanticChunker(
    embeddings=OpenAIEmbeddings(),
    breakpoint_threshold_type='percentile',   # or 'standard_deviation'
    breakpoint_threshold_amount=95
)
chunks = splitter.split_documents(docs)
```

Best for documents where topics shift gradually. More expensive since it embeds during chunking. Produces variable-size chunks — can be large when content is dense and consistent.

chunk_size vs chunk_overlap

- **chunk_size:** max size of each chunk (in characters or tokens). Too large → chunks carry irrelevant context. Too small → lose meaning. 500-1500 chars is typical.
- **chunk_overlap:** how many characters/tokens are repeated between adjacent chunks. Ensures that sentences at chunk boundaries aren't lost. Typically 10-20% of chunk_size.

Part 6 — Embeddings & Vector Databases

Embeddings convert text into dense numerical vectors. Similar texts have vectors that are close together in vector space. Vector databases store and index these vectors so you can find the most similar ones to a query very fast.

6.1 Embedding Models

```
from langchain_openai import OpenAIEmbeddings
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_google_genai import GoogleGenerativeAIEmbeddings

# OpenAI — high quality, paid
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')  # 1536 dims
embeddings = OpenAIEmbeddings(model='text-embedding-3-large')  # 3072 dims

# HuggingFace — free, runs locally
embeddings = HuggingFaceEmbeddings(
    model_name='sentence-transformers/all-MiniLM-L6-v2'  # 384 dims, fast
)

# Embed a query
vector = embeddings.embed_query('What is machine learning?')
print(len(vector))   # 1536

# Embed multiple documents
```

```
vectors = embeddings.embed_documents(['text 1', 'text 2', 'text 3'])
```

6.2 Vector Databases

Vector databases store embeddings and let you do similarity search — given a query vector, find the N most similar stored vectors. Common options: FAISS (local), Chroma (local), Pinecone (cloud), Weaviate (cloud/self-hosted).

FAISS — in-memory, fast, local

```
from langchain_community.vectorstores import FAISS

# Create from documents
vectorstore = FAISS.from_documents(chunks, embeddings)

# Save and load
vectorstore.save_local('faiss_index')
vectorstore = FAISS.load_local('faiss_index', embeddings,
                               allow_dangerous_deserialization=True)

# Similarity search
results = vectorstore.similarity_search('What is overfitting?', k=4)
for doc in results:
    print(doc.page_content[:200])
```

Chroma — persistent local vector DB

```
from langchain_chroma import Chroma

# Create and persist
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory='./chroma_db'   # saves to disk automatically
)

# Load existing
vectorstore = Chroma(persist_directory='./chroma_db', embedding_function=embeddings)
```

Pinecone — cloud, scalable

```
from langchain_pinecone import PineconeVectorStore
from pinecone import Pinecone, ServerlessSpec

pc = Pinecone(api_key='your-key')
pc.create_index('my-index', dimension=1536, metric='cosine',
                spec=ServerlessSpec(cloud='aws', region='us-east-1'))

vectorstore = PineconeVectorStore.from_documents(
    chunks, embeddings, index_name='my-index'
)
```

6.3 Similarity Search with Scores

```
# Search with relevance scores (higher = more similar)
results = vectorstore.similarity_search_with_score('What is RAG?', k=4)
for doc, score in results:
    print(f'Score: {score:.4f} | {doc.page_content[:100]}')

# Filter by metadata
results = vectorstore.similarity_search(
    'overfitting',
    k=3,
    filter={'source': 'chapter_3.pdf'}
)
```

Part 7 — Retrievers

A retriever takes a query and returns relevant documents. The simplest retriever is just a similarity search on a vector store, but LangChain provides smarter retrievers that improve result quality in different ways.

7.1 Basic Vector Store Retriever

```
# Convert any vectorstore to a retriever
retriever = vectorstore.as_retriever(
    search_type='similarity',   # or 'mmr', 'similarity_score_threshold'
    search_kwargs={'k': 4}
)

docs = retriever.invoke('What is gradient descent?')
```

7.2 MMR — Max Marginal Relevance

Basic similarity search can return redundant results — 4 very similar chunks that all say the same thing. MMR balances relevance and diversity: each result should be relevant to the query AND different from already-selected results.

```
retriever = vectorstore.as_retriever(
    search_type='mmr',
    search_kwargs={
        'k': 4,              # final number to return
        'fetch_k': 20,       # candidates to consider before diversity filtering
        'lambda_mult': 0.5   # 0 = max diversity, 1 = max relevance
    }
)
```

The algorithm: fetch fetch_k candidates by similarity, then iteratively pick the one with the best trade-off between similarity to query and dissimilarity to already-picked docs. lambda_mult=0.5 means equal weight to both.

When to use MMR: when your document chunks have a lot of repetition, or when you want to give the LLM a more varied set of context chunks rather than N near-identical ones.

7.3 Multi-Query Retriever

A single user query can be phrased in many ways, and simple similarity search finds only docs similar to that exact phrasing. Multi-query retriever uses an LLM to generate multiple reworded versions of the query, runs each one, and merges all unique results.

```
from langchain.retrievers import MultiQueryRetriever

base_retriever = vectorstore.as_retriever(search_kwargs={'k': 3})

multi_query = MultiQueryRetriever.from_llm(
    retriever=base_retriever,
    llm=llm,
    include_original=True    # also include results from original query
)

# LLM generates ~3 variations of the query automatically
docs = multi_query.invoke('How do I prevent my model from overfitting?')
# Internally generates:
# - 'What techniques reduce overfitting in ML?'
# - 'How to regularize a neural network?'
# - 'Methods to improve model generalization'
# Runs all queries, deduplicates, returns merged results
```

When to use: when users ask questions in vague or unconventional ways, or when your retrieval quality is low because queries don't match document phrasing well.

7.4 Contextual Compression Retriever

Standard retrieval returns whole chunks — but only part of a chunk may actually be relevant to the query. Contextual compression uses an LLM to extract and return only the relevant portion of each retrieved chunk, reducing noise sent to the final LLM.

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor
```

```
base_retriever = vectorstore.as_retriever(search_kwargs={'k': 4})

# Compressor — uses LLM to extract relevant parts
compressor = LLMChainExtractor.from_llm(llm)

compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=base_retriever
)

# Returns compressed, query-relevant excerpts instead of full chunks
docs = compression_retriever.invoke('What is the learning rate?')
```

When to use: when chunks are large and heterogeneous (one chunk covers many topics), or when you want to reduce token count in the final context without sacrificing relevance.

7.5 Full RAG Chain Putting It All Together

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough

retriever = vectorstore.as_retriever(search_kwargs={'k': 4})

def format_docs(docs):
    return '\n\n'.join(doc.page_content for doc in docs)

prompt = ChatPromptTemplate.from_template('''
Answer the question using only the context below.
If you don't know, say you don't know.

Context: {context}

Question: {question}
''')

rag_chain = (
    {'context': retriever | format_docs, 'question': RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

answer = rag_chain.invoke('What is the main topic of chapter 2?')
```

Interview Questions

Framework & Prompts

Q1. What is LCEL (LangChain Expression Language) and what does the | operator do?

Answer: LCEL is LangChain's composition system using the pipe operator |. It connects runnables (prompts, models, parsers, retrievers) into a pipeline where each component's output becomes the next component's input. It's inspired by Unix pipes. Benefits: lazy evaluation (nothing runs until .invoke()), built-in async and streaming support, automatic tracing with LangSmith, and easy batch processing with .batch().

Q2. What is the difference between PromptTemplate and ChatPromptTemplate?

Answer: PromptTemplate produces a plain string prompt — used with legacy LLM models (text in, text out). ChatPromptTemplate produces a list of messages (SystemMessage, HumanMessage, AIMessage) — used with chat models like GPT-4 and Claude. Chat models expect structured message lists, not raw strings. In practice, use ChatPromptTemplate for all modern models.

Q3. How does a PydanticOutputParser work and why is it useful?

Answer: PydanticOutputParser takes a Pydantic model class, generates format instructions from the model's field definitions, and adds them to the prompt so the LLM knows to output valid JSON matching that schema. When the response comes back, it parses the JSON into a typed Pydantic object. Useful because you get type-safe structured output — instead of parsing JSON strings yourself, you get a Python object with proper types and validation.

Q4. What is a partial variable in a PromptTemplate?

Answer: A partial variable is a variable that's pre-filled at template definition time, leaving fewer variables for the caller to provide. Useful for static context that doesn't change between calls — like today's date, API version, or user role. prompt.partial(date='2024-01-01') returns a new template that only needs the remaining variables. Avoids having to pass the same static value every time.

Chains

Q5. What is the difference between RunnableParallel and a SequentialChain?

Answer: SequentialChain runs components one after another — output of step N is input to step N+1. RunnableParallel runs multiple chains simultaneously on the same input and collects all results into a dict. Use sequential when each step depends on the previous one. Use parallel when you want multiple independent analyses of the same input (e.g. generate pros, cons, and use cases all at once) — parallel is faster because all chains run concurrently.

Q6. What does RunnablePassthrough do and when do you need it?

Answer: RunnablePassthrough passes input through unchanged. You need it when building parallel inputs for a chain — for example in RAG, you need both the retrieved context AND the original question passed to the prompt. {'context': retriever | format_docs, 'question': RunnablePassthrough()} routes the query to the retriever for context, but also passes the original query as 'question' unchanged.

Q7. How would you implement conditional logic in a LangChain chain?

Answer: Use RunnableBranch — it takes a list of (condition, chain) tuples and a default chain. For each invocation, it evaluates conditions in order and runs the first chain whose condition returns True. Conditions are lambda functions over the input dict. Alternatively, use RunnableLambda to write a Python function that decides which chain to invoke dynamically.

Tools & LangSmith

Q8. What does the @tool decorator do in LangChain?

Answer: It wraps a Python function into a LangChain Tool object. It reads the function name and docstring to create the tool's name and description (sent to the LLM so it knows when to use the tool), and reads the type annotations to build the JSON schema for arguments. The LLM uses the name + description to decide whether to call the tool, and the schema to know what arguments to pass.

Q9. How does LangSmith differ from just adding print statements for debugging?

Answer: Print statements show you what you added — LangSmith captures everything automatically: every prompt, every model response, every tool call, token count, latency, and cost. It gives a visual trace tree of nested chain calls so you can see exactly where a pipeline broke down. It also stores all runs so you can compare across time, build datasets from real traces, and run automated evals. Print statements disappear in production; LangSmith persists.

Document Loaders & Text Splitters

Q10. Why is RecursiveCharacterTextSplitter preferred over CharacterTextSplitter?

Answer: CharacterTextSplitter splits on one fixed separator — if you hit a chunk boundary mid-sentence, you get cut text. RecursiveCharacterTextSplitter tries a priority list of separators: paragraph break → sentence → word → character. It always uses the most natural split point available. The result is chunks that respect document structure and rarely cut mid-sentence, which produces better embeddings and retrieval.

Q11. What is chunk_overlap and why is it important?

Answer: chunk_overlap is the number of characters/tokens repeated at the end of one chunk and the start of the next. Without overlap, if a key sentence falls right at a chunk boundary, it gets split across two chunks — neither chunk has the full context. Overlap ensures continuity. Typically set to 10-20% of chunk_size. Too high wastes storage and adds noise; too low risks losing boundary context.

Q12. When would you choose semantic chunking over recursive character splitting?

Answer: Semantic chunking when you have long documents where topics shift gradually and you want chunk boundaries to align with topic changes rather than character counts. It produces cleaner chunks for retrieval since each chunk is semantically coherent. Downside: much slower (embeds every sentence during chunking) and more expensive. Recursive splitting is good enough for most use cases; semantic chunking is worth it for large knowledge bases where retrieval quality is critical.

Embeddings, VectorDB & Retrievers

Q13. What is the difference between FAISS and Pinecone?

Answer: FAISS is an in-memory vector store — fast, free, runs locally, but data is lost when the process ends (unless you save_local). Good for prototyping or small datasets. Pinecone is a managed cloud vector database — persists data, handles millions of vectors, provides filtering, scaling, and high availability out of the box, but costs money. Choose FAISS for development and small scale; Pinecone (or Weaviate/Qdrant) for production at scale.

Q14. Explain Max Marginal Relevance. What problem does it solve?

Answer: Standard k-NN similarity search can return k chunks that are all nearly identical — different paragraphs saying the same thing. This wastes the context window. MMR solves this by balancing relevance and diversity: after selecting the most relevant chunk, each subsequent selection picks the chunk that maximizes relevance to the query while minimizing similarity to already-selected chunks. The lambda_mult parameter (0-1) controls the trade-off — 0 = max diversity, 1 = max relevance.

Q15. How does a Multi-Query Retriever improve retrieval quality?

Answer: A single query phrased one way may not match the wording in the source documents. Multi-query retriever uses an LLM to generate 3-5 alternative phrasings of the original query (synonyms, different angles, more specific, more general), runs each against the vector store, and deduplicates the combined results. This significantly improves recall — even if the original query misses relevant chunks, a rephrasing may catch them.

Q16. What does a Contextual Compression Retriever do differently from a basic retriever?

Answer: A basic retriever returns whole chunks regardless of how much of the chunk is relevant. A contextual compression retriever runs each retrieved chunk through an LLM compressor that extracts only the sentences/passages actually relevant to the query. The final context passed to the answer LLM is shorter and more focused. This reduces hallucination risk (less irrelevant text confusing the model) and saves tokens. Trade-off: adds an extra LLM call per chunk, so it's slower and costs more.

Q17. Walk me through a complete RAG pipeline in LangChain from document to answer.

Answer:

- Load: PyPDFLoader / WebBaseLoader → list of Document objects
- Split: RecursiveCharacterTextSplitter → list of smaller chunks
- Embed: OpenAIEmbeddings.embed_documents() → vector for each chunk
- Store: FAISS.from_documents() or Chroma → vector index
- Retrieve: vectorstore.as_retriever() → given query, return top-k chunks
- Generate: RAG chain = {context: retriever | format_docs, question: RunnablePassthrough()} | prompt | llm | StrOutputParser()
- (Optional) Improve: swap basic retriever for MMR, Multi-Query, or Contextual Compression